

MATERIAL DE APOIO DO ALUNO

BeeCrowd com Python

Guia de referência rápida para resolver problemas: entradas, saídas, casas decimais, conversões de tipo e split de dados na mesma linha.

hello.py

```
print("Hello World!")
```

2.400+

PROBLEMAS

9

CATEGORIAS

200+

PAÍSES

judge.beecrowd.com · Python 3



Como o juiz pensa

O **BeeCrowd** (antigo URI Online Judge) é um juiz online que corrige exercícios de programação de forma automática. Você lê o enunciado de um problema, escreve o código no seu computador, testa e envia pela plataforma. Em poucos segundos, o juiz executa seu programa com os casos de teste oficiais e compara a **saída produzida** com a **saída esperada** — caractere por caractere. Não há professor olhando seu código: o que vale é o texto que aparece na tela.

Essa correção automática tem uma consequência direta para quem está começando: um programa "quase certo" é tratado como errado. Se o enunciado pede `X = 30` e o seu programa imprime `X=30` (sem espaços), o veredito será negativo. Por isso, este guia dedica atenção especial a detalhes pequenos que costumam decidir entre o Accepted e o Wrong Answer.

1

Ler o problema

Entrada, saída e exemplos definem o contrato.

2

Codificar

Escreva e teste o código localmente antes de enviar.

3

Submeter

Cole o código e selecione **Python 3** no envio.

4

Veredito

O juiz responde em segundos com o resultado.

Os vereditos que você vai encontrar

Accepted (AC)

Saída idêntica à esperada em todos os casos de teste. É o objetivo!

Wrong Answer (WA)

A saída difere do esperado: lógica errada, formatação ou tipos incorretos.

Presentation Error (PE)

A resposta está perto, mas há espaço ou quebra de linha extra na saída.

Compilation Error (CE)

O código nem roda: erro de sintaxe, como parênteses ou dois-pontos faltando.

Time Limit Exceeded (TLE)

O programa demorou demais; geralmente laço infinito ou entrada mal lida.

REGRA DE OURO

O juiz compara a saída **caractere por caractere**. Espaço a mais, letra maiúscula diferente, vírgula no lugar do ponto ou linha em branco adicional podem transformar uma solução correta em Wrong Answer. Antes de enviar, confira a saída dos exemplos do enunciado — ela é a sua referência exata de formatação.

input() sempre retorna string

No Python, a função `input()` lê uma linha digitada e **sempre devolve uma string**, mesmo que você digite apenas números. Ela faz uma pausa e espera o usuário (ou o juiz, nos casos de teste) enviar uma linha terminada por Enter. Essa simplicidade esconde a armadilha mais comum de quem está começando no BeeCrowd.

Considere o **Problema 1001**, que lê dois inteiros e imprime a soma no formato `X = soma`. O código abaixo parece correto à primeira vista — lê dois valores e "soma". Mas veja o que acontece na prática:

erro_1001.py — atenção: código com bug

```
a = input()
b = input()
print(a + b)
```

ENTRADA

10
20

SAÍDA OBTIDA

1020

VEREDITO

Wrong Answer

Esperado: X = 30

O resultado `1020` revela o problema: como `a` e `b` são strings, o operador `+` faz **concatenação** (junta os textos `'10'` e `'20'`), e não soma numérica. O Python até ajuda a perceber isso: `type(input())` retorna `<class 'str'>` para qualquer entrada. Em outras palavras, para o interpretador, digitar `10` equivale a digitar `dez` — é texto.

console — provando o tipo

```
x = input()    # usuário digita 10
print(type(x)) # <class 'str'>
print(type(int(x))) # <class 'int'>
```

ARMADILHA CLÁSSICA

Toda entrada do BeeCrowd chega como **texto**. Antes de fazer contas, você precisa converter para número com `int()` ou `float()`. Somar strings concatena; subtrair strings, aí sim, dá erro de execução em tempo de submissão.

Transformando texto em número

Se `input()` devolve texto, o próximo passo óbvio é converter esse texto no tipo certo. O Python oferece três funções de conversão básicas: `int()` para inteiros, `float()` para números com casas decimais e `str()` para transformar qualquer valor em texto. A escolha

depende do problema: idade e quantidade pedem `int()`; preço, salário e média pedem `float()`.

Há três comportamentos que você precisa memorizar, porque eles aparecem em provas e em submissões o tempo todo. Primeiro: `int()` aplicado a um float **trunca** (corta as casas decimais) em vez de arredondar. Segundo: `int()` aplicado a uma string com ponto, como `'3.5'`, gera `ValueError` — é preciso passar por `float()` primeiro. Terceiro: `float()` aceita tanto `'3.5'` quanto `'3'`, então é a conversão mais tolerante para entradas decimais.

Expressão	Resultado	O que acontece
<code>int("10")</code>	10	Texto com número inteiro vira int — uso padrão do BeeCrowd.
<code>float("3.5")</code>	3.5	Texto com ponto decimal vira float.
<code>str(10)</code>	"10"	Número vira texto — útil para concatenar em mensagens.
<code>int(3.9)</code>	3	Trunca , não arredonda: as decimais são descartadas.
<code>int("3.5")</code>	ValueError	String com ponto não converte direto para int.
<code>float(10)</code>	10.0	Inteiro vira float — o ponto aparece na exibição.

Com a conversão no lugar, o Problema 1001 finalmente fica correto. Note que a saída também mudou: o enunciado pede o texto `X =` antes do valor, exatamente com esses espaços. Ler com atenção o formato da saída faz parte de resolver o problema.

```
solucao_1001.py - Extremamente Básico

a = int(input())
b = int(input())
print("X =", a + b)
```

ENTRADA

10
20

SAÍDA

X = 30

VEREDITO

Accepted

1º problema resolvido!

DICAS DE CONVERSÃO

Use `int(input())` quando o problema fala em valores inteiros e `float(input())` quando mencionar ponto flutuante, dupla precisão ou casas decimais. Na dúvida, `float()` é mais permissivo — mas lembre de formatar a saída depois, como você verá na seção 5.

4 SAÍDAS `print()` e f-strings

Se `input()` é a porta de entrada, `print()` é a porta de saída — e é por ela que o juiz avalia você. A função aceita vários valores separados por vírgula e os imprime na mesma linha, **separados por um espaço**. Esse espaço automático é o responsável por `print("X =", 30)` produzir exatamente `X = 30`, com um espaço antes e depois do sinal de igual.

Três formas de montar a mesma saída

Tomando o **Problema 1004** (ler dois inteiros e imprimir o produto no formato `PROD = valor`) como exemplo, existem três caminhos comuns. Todos produzem `PROD = 27`; o que muda é a legibilidade e a flexibilidade.

saída_1004.py — três estilos

```
a = int(input())
b = int(input())

# 1) Vírgulas: print insere espaço entre os itens
print("PROD =", a * b)

# 2) Concatenação: precisa converter cada número com str()
print("PROD = " + str(a * b))

# 3) f-string: o valor entra direto nas chaves (recomendado)
print(f"PROD = {a * b}")
```

ENTRADA

3
9

SAÍDA (as 3 formas)

PROD = 27

A **f-string** é a forma recomendada e a mais usada neste guia. Basta colocar a letra `f` antes das aspas e escrever expressões entre chaves: `f"PROD = {a * b}"`. Dentro das chaves cabe qualquer expressão Python válida — `{a * b}`, `{(a + b) / 2}`, `{a: 2}` — e o

resultado é convertido para texto automaticamente. Ela combina com formatação de casas decimais, que é o assunto da próxima seção.

CUIDADO COM A CONCATENAÇÃO

No estilo 2, escrever `"PROD = " + a * b` sem o `str()` gera `TypeError`: Python se recusa a somar string com número. Se vir esse erro na submissão, procure um `+` entre textos e números — e troque por vírgula ou f-string.



PRECISÃO

Casas decimais com .2f e .4f

Muitos problemas pedem a saída com um número fixo de casas decimais: média com 1 casa, área com 4 casas, valor a pagar com 2 casas. O Python resolve isso dentro da própria f-string, com o especificador `:.Nf`, onde N é a quantidade de casas. Ele **arredonda** o valor para exibir exatamente N dígitos após o ponto — e o ponto é o padrão americano, que é o que o BeeCrowd espera.

O **Problema 1002** pede a área de um círculo: $A = 3.14159 \times R^2$, com R de dupla precisão e saída com 4 casas decimais, no formato `A=valor`. Repare nos detalhes de formatação do enunciado: não há espaço entre `A` e `=`, e o valor usa ponto. O código precisa respeitar isso à risca.

solucao_1002.py — Área do Círculo

```
r = float(input())
a = 3.14159 * r ** 2
print(f"A={a:.4f}")
```

ENTRADA

2.00

SAÍDA

A=12.5664

VEREDITO

Accepted

Sem espaço após o =

round() muda o valor; formatação muda o texto

Existe uma diferença sutil entre a função `round()` e o especificador `:.2f`. O `round(5.5, 2)` devolve o **próprio número** arredondado — que, por ser 5.5, exibe como `5.5`, sem o zero final. Já `f"{5.5:.2f}"` devolve um **texto formatado** com exatamente duas casas: `5.50`. Para o BeeCrowd, o que conta é o texto final; por isso, prefira sempre a formatação dentro da f-string.

Expressão	Resultado	Observação
<code>f"{5.5:.2f}"</code>	5.50	Texto com 2 casas — zero à direita é mantido.
<code>round(5.5, 2)</code>	5.5	Número arredondado — exibe sem o zero final.
<code>f"{12.56637:.4f}"</code>	12.5664	Arredonda o 5º dígito para exibir 4 casas.
<code>f"{15.5:.2f}"</code>	15.50	Padrão do "VALOR A PAGAR: R\$ 15.50".
<code>f"{0.33333:.1f}"</code>	0.3	Uma casa: 0.33 vira 0.3 na exibição.

PONTO OU VÍRGULA?

O BeeCrowd usa o padrão americano: **ponto** como separador decimal. Nunca troque o `.2f` por `,2f` — isso é erro de sintaxe. E se a entrada vier com vírgula ("1,50"), trate-a como texto e substitua antes de converter: `float(entrada.replace(",", "."))`.

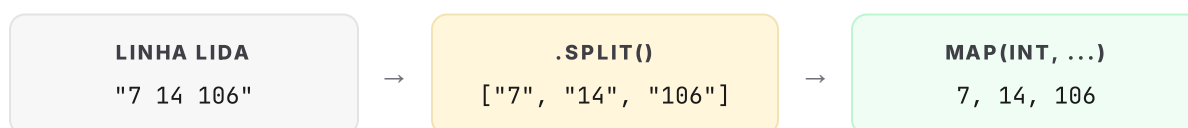


SPLIT DE DADOS

Vários valores na mesma linha

Até aqui, cada problema leu um valor por linha. Mas muitos enunciados entregam **vários dados na mesma linha, separados por espaço** — por exemplo, o Problema 1013 traz "7 14 106" em uma única linha. Como o `input()` lê a linha inteira como uma string, você recebe `'7 14 106'`: um texto só. A ferramenta para separá-lo é o método `split()`.

Sem argumentos, `split()` corta a string nos espaços (um ou vários) e devolve uma **lista de strings**. Só que uma lista de textos ainda não serve para contas — cada elemento precisa ser convertido para número. Para transformar todos de uma vez, combina-se o `map()`: ele aplica `int` a cada item da lista. Por fim, o desempacotamento `a, b, c = ...` distribui os três valores em três variáveis, em ordem.



● ● ● leitura_linha.py — a receita completa

```
linha = input()          # "7 14 106"
partes = linha.split()   # ["7", "14", "106"]
a = int(partes[0])       # 7 (índice começa em 0)
b = int(partes[1])       # 14
c = int(partes[2])       # 106

# Ou tudo em uma linha só, com map + desempacotamento:
a, b, c = map(int, input().split())
```

Com a leitura resolvida, o **Problema 1013** (O Maior) pede o maior de três valores inteiros distintos. A fórmula matemática clássica usa `abs()`: o maior entre x e y é $(x + y + |x - y|) / 2$. Aplicando duas vezes, você cobre os três valores. A saída exige o texto `eh o maior` — escrito sem acento, como no enunciado original.

● ● ● solucao_1013.py — O Maior

```
a, b, c = map(int, input().split())

maior_ab = (a + b + abs(a - b)) / 2
maior    = (maior_ab + c + abs(maior_ab - c)) / 2

print(f"{maior:.0f} eh o maior")

# Alternativa enxuta: o Python já tem a função pronta
# print(f"{max(a, b, c)} eh o maior")
```

ENTRADA

7 14 106

SAÍDA

106 eh o maior

VEREDITO

Accepted

split + map na prática

POR QUE map E NÃO UM LAÇO?

Você poderia converter item por item com índices, e para 2 ou 3 valores isso funciona bem. O padrão `map(int, input().split())` é mais curto, menos propenso a erro de digitação e continua funcionando se o problema tiver 4, 5 ou 10 valores na mesma linha — basta ajustar as variáveis do desempacotamento. Acostume-se com ele: é a receita oficial de entrada do BeeCrowd em Python.



SEPARADORES

split com vírgula, ponto e vírgula

O `split()` sem argumentos assume espaço como separador. Alguns problemas, porém, entregam dados separados por **vírgula** ou **ponto e vírgula** — comum em listas de nomes

com valores, como `Maria,20` ou `João;15.50`. Nesses casos, passe o separador como argumento: `split(",")` ou `split(";")`. O método devolve a lista de strings, e a conversão segue igual à da seção anterior.

```
separadores.py

# Entrada: "Maria,20"
nome, idade = input().split(",")
idade = int(idade)

# Entrada: "João;15.50"
nome, nota = input().split(";")
nota = float(nota)
```

Chamada	Resultado	Quando usar
<code>input().split()</code>	<code>["7", "14", "106"]</code>	Valores separados por espaço (o mais comum).
<code>input().split(",")</code>	<code>["Maria", "20"]</code>	Dados separados por vírgula.
<code>input().split(";")</code>	<code>["João", "15.50"]</code>	Dados separados por ponto e vírgula.

CUIDADO COM O NÚMERO DE COLUNAS

Se a linha tem 3 informações e você desempacota em 2 variáveis, o Python devolve `ValueError: too many values to unpack`. Conte as colunas do exemplo de entrada do enunciado antes de escrever o desempacotamento — ou use índices, como no exemplo completo da próxima seção.



EXEMPLO COMPLETO

Tudo junto: Problema 1010

O **Problema 1010** (Cálculo Simples) junta tudo o que este guia apresentou: duas linhas de entrada, múltiplos dados por linha, conversões de tipos diferentes e saída formatada com casas decimais. O enunciado diz: cada linha traz o código do produto (inteiro), o número de unidades (inteiro) e o preço por unidade (com duas casas decimais). Calcule o valor total a pagar.

A leitura usa `split()` e índices — uma abordagem segura quando a linha mistura tipos diferentes. Como o código do produto (posição 0) não entra no cálculo, ele simplesmente fica de fora das contas: pegamos `linha1[1]` e `linha1[2]`. As unidades pedem `int()`; os preços, `float()`. A saída segue o formato exato do enunciado: `VALOR A PAGAR: R$` seguido do total com `:.2f`.

```

linha1 = input().split() # ["12", "1", "5.30"]
linha2 = input().split() # ["16", "2", "5.10"]

q1, p1 = int(linha1[1]), float(linha1[2])
q2, p2 = int(linha2[1]), float(linha2[2])

total = q1 * p1 + q2 * p2

print(f"VALOR A PAGAR: R$ {total:.2f}")

```

ENTRADA

```

12 1 5.30
16 2 5.10

```

SAÍDA

VALOR A PAGAR: R\$ 15.50

VEREDITO**Accepted**

Split, índices, conversão e .2f

Vale a pena conferir a aritmética para enxergar o caminho completo: $1 \times 5.30 = 5.30$ e $2 \times 5.10 = 10.20$, somando 15.50 . Sem o `:.2f`, o Python exibiria 15.5 — e o veredito seria Wrong Answer, porque falta o zero. Este exemplo resume a disciplina que o BeeCrowd ensina: **ler no formato que chega, calcular com os tipos certos e imprimir no formato que pedem**.

**DEBUG**

Os 6 erros que geram Wrong Answer

Antes de culpar o juiz (todos já fizemos isso), passe por esta lista. Ela concentra os erros que respondem pela esmagadora maioria dos WAs de quem está começando em Python. Cada item traz o sintoma e a correção — se a sua submissão foi rejeitada, provavelmente está em um deles.

1**Somar sem converter**

`a + b` com strings concatena em vez de somar: `10` e `20` viram `1020`. Sempre converta com `int()` ou `float()` logo após o `input()`.

2**Mensagem ou texto extra na saída**

Escrever `"Digite um número:"` dentro do `input()` ou imprimir `"O resultado é:"` além do valor polui a saída. O juiz espera **apenas** o que o enunciado pede — nada mais.

3

Casas decimais fora do formato

Imprimir `15.5` quando o problema pede `15.50`. Use `f"{valor:.2f}"` (ou `:.4f`, conforme o enunciado) para fixar as casas.

4

Espaço ou sinal de igual diferente

O problema 1002 pede `A=12.5664` (sem espaço); o 1001 pede `X = 30` (com espaços). Copie o formato do exemplo de saída literalmente, caractere por caractere.

5

int() em número com casas decimais

`int("5.30")` gera `ValueError` na submissão. Quando o dado tem ponto, converta com `float()`; use `int()` só para valores genuinamente inteiros.

6

Esquecer o f antes da string

Sem o `f`, as chaves viram texto comum: `"PROD = {a * b}"` imprime literalmente `PROD = {a * b}`. Confira o `f` antes das aspas sempre que usar chaves.

COMO DEBUGAR RÁPIDO

Rode o programa localmente com a entrada de exemplo do enunciado e compare a sua saída com a esperada, caractere por caractere — inclusive espaços. Um truque útil: imprima `repr(sua_saida)` no terminal para enxergar espaços e quebras de linha invisíveis.

10

ANTES DE ENVIAR**Checklist de submissão**

Submeter sem revisar é apostar. Este checklist leva menos de um minuto e elimina os erros da seção anterior antes que eles custem uma submissão. Com o tempo, os itens viram reflexo — e as suas taxas de Accepted na primeira tentativa vão subir visivelmente.



Li a entrada inteira? Quantos valores, em quais linhas, inteiros ou decimais, separados por espaço ou outro símbolo.



Converti os tipos certos? `int()` para inteiros, `float()` para decimais — e `split()` / `map()` quando há vários valores por linha.



Copiei o formato da saída? Textos, espaços, sinal de igual e maiúsculas exatamente como no exemplo de saída do enunciado.



Formatei as casas decimais? `.2f`, `.4f` ... conforme pedido, com ponto e não vírgula.



Testei com o exemplo do enunciado? Entrada e saída conferidas caractere por caractere, sem mensagens extras.



Selecionei Python 3 no envio? O dropdown de linguagem na página de submissão precisa estar em Python 3.



PRÓXIMOS PASSOS

Roteiro de prática

A categoria **Iniciante** do BeeCrowd vai do problema 1000 ao 1052 aproximadamente e foi desenhada para exercitar exatamente as técnicas deste guia: leitura, conversão, formatação e split. Resolva na ordem — cada problema acrescenta um detalhe novo em relação ao anterior. Quando os seis primeiros estiverem verdes, você já domina o ciclo completo de uma submissão.

1000

Hello World!

Primeiro print — sem entrada, só a mensagem exata.

1001

Extremamente Básico

Ler dois inteiros, converter com `int()` e somar.

1002

Área do Círculo

`float()` na entrada e `.4f` na saída.

1004

Produto Simples

f-string para montar "PROD = valor".

1010

Cálculo Simples

Duas linhas, split com índices e total com `.2f`.

1013

O Maior

`map(int, input().split())` e a fórmula do maior.

Depois, avance para os **desafios intermediários**: o 1003 (Soma Simples) treina o split de dois valores; o 1005 (Média 1) mistura pesos com float; o 1006 e o 1007 (Média 2 e Diferença) combinam várias leituras; e o 1008 (Salário) fecha o ciclo com cálculo e formatação monetária. Faça um por vez, sempre com o checklist da seção 10 — e comemore cada Accepted: eles são a prova de que a técnica virou hábito.

SUA VEZ

Bora praticar!

Crie sua conta, resolva o problema 1000 e siga o roteiro de prática. Um problema por dia já transforma a curva de aprendizado.



```
# judge.beecrowd.com · Python 3
```

```
a, b = map(int, input().split())  
print(f"X = {a + b}")
```

```
# Agora é com você.
```

judge.beecrowd.com

Guia de referência — BeeCrowd com Python · Material de apoio do aluno